



Guía de GitHub para Estudiantes

Josué Heber Fabián Martínez

Moisés Velasco Lozano

Material de apoyo — Geociencias / Ingeniería Petrolera

Esta guía está pensada para estudiantes que están dando sus primeros pasos con GitHub como herramienta de control de versiones y colaboración, aplicada al trabajo con notebooks de Python (Jupyter), scripts, reportes y proyectos de análisis de datos.

Nota para el estudiante: todas las imágenes de esta guía son diagramas ilustrativos elaborados para este documento (no son capturas de pantalla reales de GitHub); la interfaz real puede variar ligeramente en apariencia, aunque los pasos y campos descritos se mantienen vigentes.

Índice

1	¿Qué es GitHub y por qué usarlo?	2
2	Crear una cuenta en GitHub	2
3	Crear un repositorio	3
4	Subir un archivo	4
4.1	Desde el navegador (sin usar Git)	4
4.2	Desde tu computadora con Git (línea de comandos)	5
5	Dar acceso a otros usuarios (colaboradores)	6
6	Instalar GitHub Desktop	6
7	Dar seguimiento a versiones del código en un Notebook	7
7.1	Con GitHub Desktop (recomendado para principiantes)	8
7.2	Consultar el historial de versiones	8
8	Buenas prácticas para trabajo en equipo	8

1 ¿Qué es GitHub y por qué usarlo?

Git es un sistema de control de versiones: permite guardar el historial completo de los cambios hechos a un conjunto de archivos (código, notebooks, documentos), pudiendo regresar a versiones anteriores en cualquier momento. **GitHub** es una plataforma en la nube que aloja repositorios de Git, y añade herramientas de colaboración: control de acceso, revisión de cambios, issues, y una interfaz web para trabajar sin necesidad de usar la línea de comandos.

Para un estudiante, GitHub es útil para:

- Guardar una copia respaldada en la nube de tareas, proyectos y notebooks.
- Llevar un historial de versiones del código: qué cambió, cuándo y por qué.
- Compartir el trabajo con compañeros, profesores o asesores de tesis.
- Colaborar en un mismo proyecto sin sobrescribir el trabajo de otros.
- Construir un portafolio público de proyectos (muy valorado al buscar prácticas profesionales o empleo).

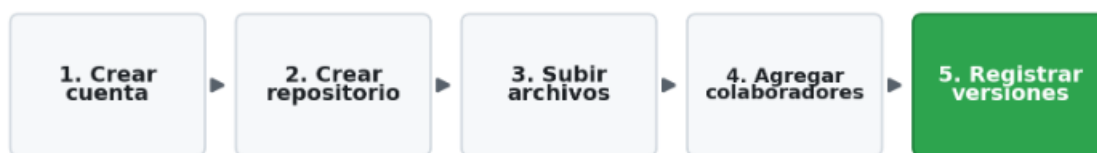


Figura 1: Flujo general que se cubre en esta guía: de crear la cuenta a registrar versiones del trabajo.

2 Crear una cuenta en GitHub

Sigue estos pasos para crear tu cuenta:

1. Entra a github.com desde tu navegador.
2. Haz clic en **Sign up** (Registrarse), en la esquina superior derecha.
3. Ingresa tu correo electrónico (se recomienda usar tu correo institucional, si tu universidad participa en el **GitHub Student Developer Pack**).
4. Crea una contraseña segura y elige un nombre de usuario. Este nombre será parte de la dirección de tus repositorios (por ejemplo, github.com/tu-usuario), así que conviene elegir uno profesional y fácil de recordar.
5. Completa la verificación que te solicite GitHub y confirma tu correo electrónico desde el enlace que te enviarán.

El diagrama muestra una ventana de navegador con la URL `github.com/signup`. El título principal es "Crea tu cuenta". Hay tres campos de entrada: "Correo electrónico" con el valor `tu-correo@universidad.edu`, "Contraseña" con caracteres ocultos por puntos, y "Nombre de usuario" con el valor `tu-usuario`. Debajo de los campos hay un botón verde que dice "Sign up".

Figura 2: Diagrama ilustrativo del formulario de registro de una cuenta nueva en GitHub.

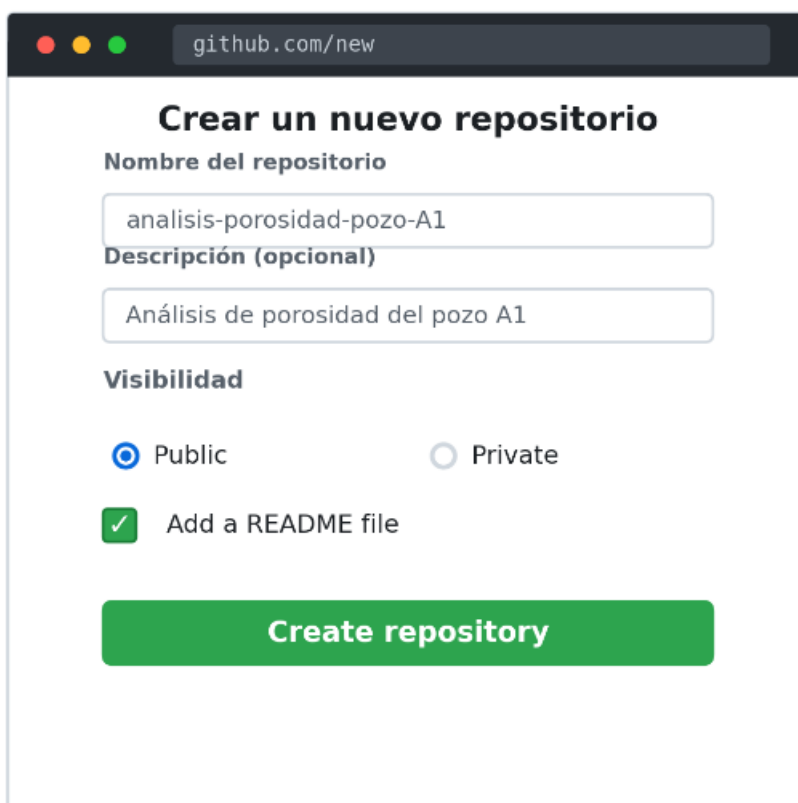
Nota para el estudiante: como estudiante, puedes solicitar el *GitHub Student Developer Pack* en `education.github.com`, que da acceso gratuito a herramientas adicionales (GitHub Pro, Copilot, entre otras) usando tu correo universitario.

3 Crear un repositorio

Un **repositorio** (o “repo”) es la carpeta de proyecto donde vivirán tus archivos y todo su historial de cambios. Cada proyecto (por ejemplo, cada materia o cada trabajo de investigación) debería tener su propio repositorio.

1. Inicia sesión en GitHub y haz clic en el ícono `+` de la esquina superior derecha, y selecciona **New repository**.
2. Escribe un nombre descriptivo para el repositorio (por ejemplo, `analisis-porosidad-pozo-A1`), evitando espacios y acentos.
3. Agrega una breve descripción del proyecto (opcional, pero recomendable).
4. Elige la visibilidad: **Public** (cualquiera puede verlo, ideal para portafolio) o **Private** (solo tú y las personas que invites pueden verlo, recomendable para tareas o datos sensibles).

5. Marca la casilla **Add a README file**: este archivo describe el proyecto y aparecerá como portada del repositorio.
6. Haz clic en **Create repository**.



The image shows a web browser window with the address bar displaying 'github.com/new'. The main heading is 'Crear un nuevo repositorio'. Below this, there are three sections: 'Nombre del repositorio' with a text input field containing 'analisis-porosidad-pozo-A1'; 'Descripción (opcional)' with a text input field containing 'Análisis de porosidad del pozo A1'; and 'Visibilidad' with two radio buttons, 'Public' (selected) and 'Private'. Below the visibility options is a checked checkbox labeled 'Add a README file'. At the bottom is a large green button labeled 'Create repository'.

Figura 3: Diagrama ilustrativo del formulario de creación de un nuevo repositorio.

4 Subir un archivo

4.1 Desde el navegador (sin usar Git)

Es la forma más rápida para subir un archivo puntual, como un notebook o un PDF, sin instalar nada:

1. Entra al repositorio y haz clic en **Add file** → **Upload files**.
2. Arrastra el archivo (por ejemplo, tu `Notebook.ipynb`) a la zona indicada, o haz clic para elegirlo desde tu computadora.
3. Escribe un breve mensaje describiendo el cambio (el **commit message**), por ejemplo: "Agregar notebook de análisis de porosidad".

4. Haz clic en **Commit changes** para confirmar la subida.



Figura 4: Diagrama ilustrativo de la pantalla para subir archivos desde el navegador.

4.2 Desde tu computadora con Git (línea de comandos)

Para proyectos que vas actualizando con frecuencia, conviene trabajar con Git localmente:

```
git clone https://github.com/tu-usuario/tu-repositorio.git
```

```
cd tu-repositorio
```

```
git add Notebook.ipynb
```

```
git commit -m "Agregar notebook de análisis de porosidad"
```

```
git push origin main
```

Nota para el estudiante: git add prepara los archivos que quieres guardar en el historial; git commit guarda esa versión con un mensaje; git push la sube a GitHub. La sección 7 de esta guía explica este flujo con más detalle.

5 Dar acceso a otros usuarios (colaboradores)

Cuando un repositorio es privado, solo tú puedes verlo hasta que invites a otras personas. Incluso en repositorios públicos, dar acceso de **colaborador** es necesario para que otra persona pueda subir cambios directamente (sin colaborador, solo puede sugerir cambios mediante un *pull request*).

1. Entra al repositorio y ve a la pestaña **Settings** (Configuración).
2. En el menú lateral, selecciona **Collaborators** (Colaboradores).
3. Haz clic en **Add people** y escribe el nombre de usuario o correo de la persona (por ejemplo, un compañero de equipo o tu profesor/asesor).
4. Elige el nivel de permiso: **Read** (solo lectura), **Write** (puede subir cambios) o **Admin** (control total, incluyendo configuración del repositorio).
5. La persona invitada recibirá una notificación por correo y deberá aceptar la invitación para tener acceso.

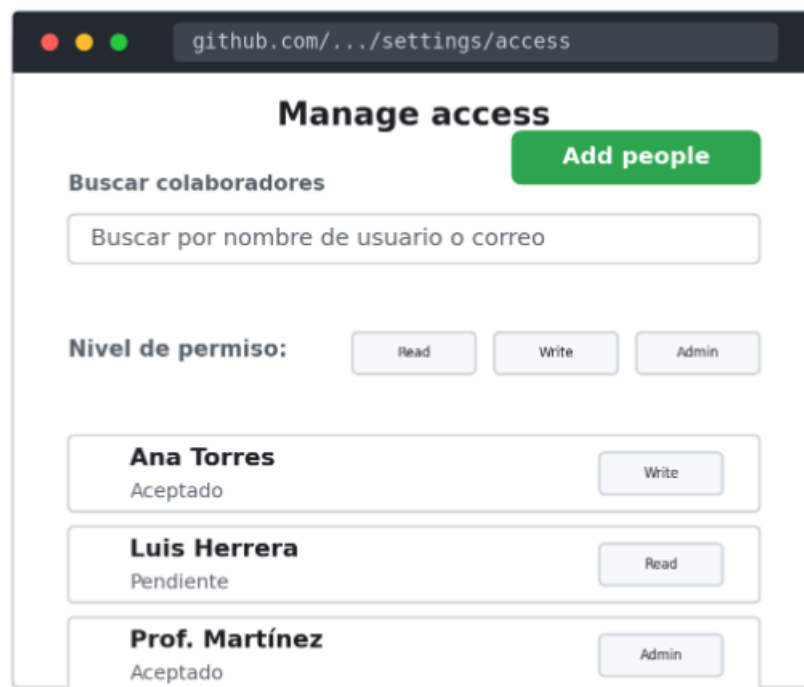


Figura 5: Diagrama ilustrativo de la sección de colaboradores en la configuración del repositorio.

6 Instalar GitHub Desktop

GitHub Desktop es una aplicación gráfica (sin línea de comandos) que facilita clonar repositorios, ver los cambios realizados, y hacer *commit* / *push* / *pull*. Es una excelente opción para quienes se están iniciando con Git.

1. Entra a `desktop.github.com` desde tu navegador.
2. Descarga el instalador correspondiente a tu sistema operativo (Windows o macOS).
3. Ejecuta el instalador y sigue las instrucciones en pantalla.
4. Al abrir la aplicación por primera vez, inicia sesión con tu cuenta de GitHub (**Sign in to GitHub.com**).
5. Usa **File** → **Clone repository** para descargar a tu computadora una copia local de un repositorio existente, o **File** → **Add local repository** si ya tienes una carpeta que quieres conectar a Git.

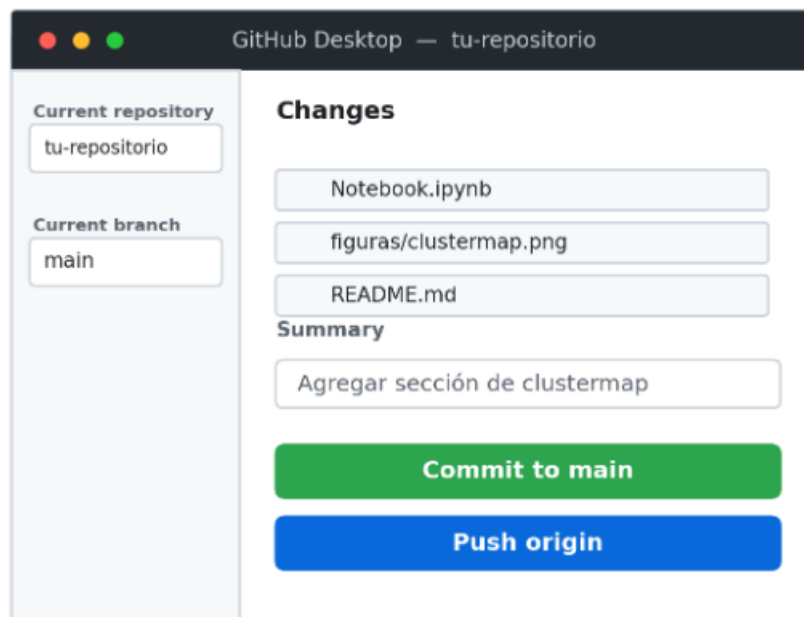


Figura 6: Diagrama ilustrativo de la ventana principal de GitHub Desktop.

Nota para el estudiante: en Windows, GitHub Desktop instala automáticamente Git; no es necesario instalarlo por separado a menos que también quieras usarlo desde la terminal.

7 Dar seguimiento a versiones del código en un Notebook

Uno de los mayores beneficios de usar GitHub para un notebook de Jupyter es poder ver **qué cambió, cuándo y quién lo cambió** a lo largo de un proyecto (por ejemplo, entre distintas entregas de una tarea o distintas versiones de un análisis).



Figura 7: Flujo básico de trabajo con Git: editar, preparar (add), guardar versión (commit) y subir (push).

7.1 Con GitHub Desktop (recomendado para principiantes)

1. Edita y guarda tu `Notebook.ipynb` normalmente en Jupyter.
2. Abre GitHub Desktop: la aplicación detecta automáticamente los archivos que cambiaron, en la sección **Changes**.
3. Escribe un mensaje breve y descriptivo en el campo **Summary** (por ejemplo, "**Agregar sección de clustermmap**").
4. Haz clic en **Commit to main** para guardar esa versión en el historial local.
5. Haz clic en **Push origin** para subir los cambios a GitHub.

7.2 Consultar el historial de versiones

En la página del repositorio en GitHub, haz clic en el ícono de reloj (**History**) o en el número de **commits** para ver la lista completa de versiones guardadas. Al abrir un commit específico, GitHub muestra exactamente qué celdas o líneas cambiaron respecto a la versión anterior.

Nota para el estudiante: los archivos `.ipynb` incluyen metadatos y las salidas de las celdas (imágenes, texto) en el mismo archivo, por lo que el historial de cambios (*diff*) puede verse menos claro que en un script `.py`. Para revisiones más limpias, algunos equipos limpian las salidas de las celdas (Cell → All Output → Clear) antes de subir el notebook.

8 Buenas prácticas para trabajo en equipo

- Escribe mensajes de **commit** claros y en tiempo presente (por ejemplo, "**Agregar**" en vez de "**Agregado**"), describiendo qué cambia y no solo "**cambios**".
- Haz commits pequeños y frecuentes en vez de un solo commit gigante al final del proyecto: facilita identificar en qué momento se introdujo un error.

- Antes de empezar a trabajar, actualiza tu copia local con **Fetch origin** / **Pull** para traer los cambios más recientes de tus compañeros.
- Usa un archivo **README.md** para explicar de qué trata el proyecto, cómo ejecutarlo y qué contiene cada carpeta.
- Evita subir archivos muy pesados (bases de datos grandes, videos) directamente al repositorio; considera un archivo **.gitignore** para excluir carpetas temporales o archivos generados automáticamente.
- Si dos personas editan el mismo notebook al mismo tiempo, es común que ocurra un **conflicto de fusión** (*merge conflict*); en ese caso, es mejor coordinarse por equipo para no perder trabajo.

Con estos pasos, un estudiante puede tener su propio flujo de trabajo ordenado: crear un repositorio por proyecto, registrar el avance con commits descriptivos, y compartir el trabajo con compañeros o profesores de forma controlada y con historial completo.